

Root Finding I: Bracketing Methods

Bisection and false position

Sharareh Sayyad

Washington State University

October 5, 2026

Outline

The root-finding problem

The bisection method

The false position method

Comparing and stopping

Summary

The Illinois method

Root finding: solve $f(x) = 0$

A **root** (or zero) of f is a number r with $f(r) = 0$.

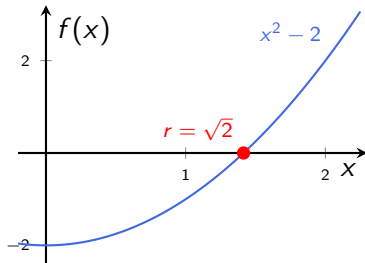
Almost any equation can be written this way:

► $x^2 = 2 \implies f(x) = x^2 - 2$

► $\cos x = x \implies f(x) = \cos x - x$

The problem

Most equations have **no formula** for the root.
We compute a sequence $x_1, x_2, \dots \rightarrow r$ and stop when it is close enough.



Today: methods that **never fail** once we have a starting interval.

Bracketing: the Intermediate Value Theorem

Intermediate Value Theorem

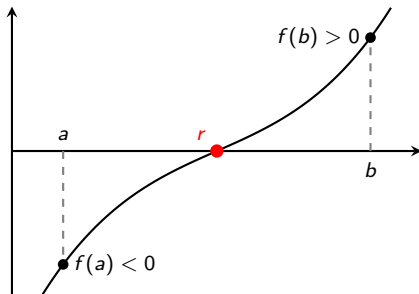
If f is **continuous** on $[a, b]$ and $f(a)$, $f(b)$ have **opposite signs**, then f has at least one root in (a, b) .

Such an interval $[a, b]$ is called a **bracket**.

Test for a bracket

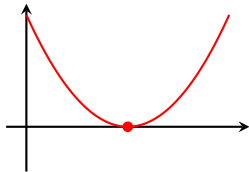
$f(a)$ and $f(b)$ have opposite signs, i.e.
`np.sign(f(a)) != np.sign(f(b))`.

Intuition: a continuous curve cannot get from below the axis to above it without crossing it.



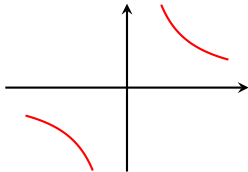
Bracketing: what can go wrong

Both conditions of the theorem matter. Always **plot f first**.



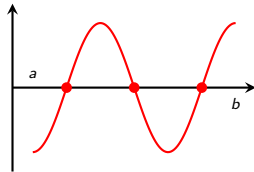
Root, no sign change

$f(x) = (x - 1)^2$ touches the axis at $x = 1$ but never changes sign: there is no bracket, so bracketing methods cannot find this root.



Sign change, no root

$f(x) = 1/x$ on $[-1, 1]$ changes sign, but f is not continuous. Bisection happily “converges” to the jump at $x = 0$.



Several roots

With three roots in $[a, b]$ there is still a sign change, and the method finds *one* of them. Which one depends on the bracket.

Bisection: halve the bracket

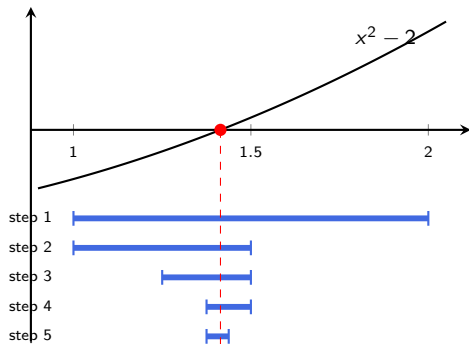
Start with a bracket $[a, b]$. Repeat:

1. Take the midpoint $c = \frac{a+b}{2}$.
2. If $f(a)$ and $f(c)$ have opposite signs, the root is in $[a, c]$: set $b \leftarrow c$.
3. Otherwise it is in $[c, b]$: set $a \leftarrow c$.

Invariant

After every step, $[a, b]$ is **still a bracket**, and it is **half as wide**.

Like guessing a number between 1 and 100 with “higher / lower” answers.



Each bracket is half the previous one and always contains $\sqrt{2}$.

Bisection in Python

```
def bisection(f, a, b, tol=1e-8, maxit=100):
    fa = f(a)
    if np.sign(fa) == np.sign(f(b)):
        raise ValueError("not a bracket")
    for n in range(maxit):
        c = a + (b - a)/2      # midpoint
        fc = f(c)              # the only new f-eval
        if (b - a)/2 < tol:    # error bound < tol
            return c
        if np.sign(fc) == np.sign(fa):
            a, fa = c, fc      # root in [c, b]
        else:
            b = c              # root in [a, c]
    return c
```

Floating point details

$a + (b-a)/2$: always lands inside $[a, b]$. `np.sign` instead of `fa*fc < 0`: the product can underflow to 0.

Why not stop when $f(c) = 0$?

- ▶ $f(c)$ is itself rounded, so an **exact** 0 almost never happens: the test is too strong.
- ▶ If it does happen, nothing breaks: $\text{sign}(0) \neq \text{sign}(f(a))$, so we keep $[a, c]$, which still contains the root c .

Why not $|f(c)| < \text{tol}$ either?

That measures the **residual**, not the **error**. For $f(x) = (x - 1)^{10}$: $|f(1.1)| = 10^{-10}$, yet 1.1 is 0.1 away from the root.

Bisection gives a true error bound for free: stop when $(b - a)/2 < \text{tol}$.

Example: $\sqrt{2}$ by bisection, $f(x) = x^2 - 2$ on $[1, 2]$

n	a	b	c	$f(c)$	$ c - \sqrt{2} $
1	1.000000	2.000000	1.500000	+0.2500	8.6×10^{-2}
2	1.000000	1.500000	1.250000	-0.4375	1.6×10^{-1}
3	1.250000	1.500000	1.375000	-0.1094	3.9×10^{-2}
4	1.375000	1.500000	1.437500	+0.0664	2.3×10^{-2}
5	1.375000	1.437500	1.406250	-0.0225	8.0×10^{-3}
6	1.406250	1.437500	1.421875	+0.0217	7.7×10^{-3}
7	1.406250	1.421875	1.414062	-0.0004	1.5×10^{-4}
8	1.414062	1.421875	1.417969	+0.0106	3.8×10^{-3}

- ▶ The **sign** of $f(c)$ decides which half to keep; its size is ignored.
- ▶ The error does **not** decrease every step: step 7 is lucky, step 8 is worse.
- ▶ What *does* shrink every step is the **bracket width**: $1, \frac{1}{2}, \frac{1}{4}, \dots$

How fast? A guaranteed error bound

After n steps the midpoint c_n satisfies

$$|c_n - r| \leq \frac{b-a}{2^n}.$$

- ▶ Linear convergence with $\rho = \frac{1}{2}$: one **bit** per step, one decimal digit every $\log_2 10 \approx 3.3$ steps.
- ▶ Known **before** we start, whatever f is.

Number of steps for tolerance tol

$$\frac{b-a}{2^n} \leq \text{tol} \iff n \geq \log_2 \frac{b-a}{\text{tol}}$$

Example

$[1, 2]$, $\text{tol} = 10^{-6}$: $n \geq \log_2 10^6 \approx 19.9$, so **20 steps**.

Cost in Big- O terms

One f -evaluation per step, so cost = number of steps:

$$n = \underbrace{\log_2(b-a)}_{\text{constant}} + \log_2 \frac{1}{\text{tol}}$$

$$\implies \text{cost} = O(\log(1/\text{tol})).$$

10 times more accuracy costs only $\log_2 10 \approx 3.3$ more steps.

False position: use the secant line

Bisection ignores the *values* of f . If $|f(a)|$ is small, the root is probably close to a .

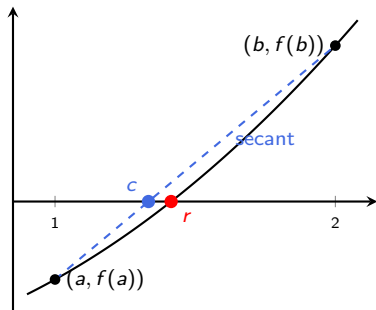
Idea: replace f by the straight line through $(a, f(a))$ and $(b, f(b))$, and take its zero:

$$c = b - f(b) \frac{b - a}{f(b) - f(a)} = \frac{a f(b) - b f(a)}{f(b) - f(a)}$$

Then keep the half with the sign change, exactly as in bisection.

In code

Only the line for c changes (and we keep f_b too). The bracket is still guaranteed.



For $x^2 - 2$ on $[1, 2]$:

$$c = \frac{1 \cdot 2 - 2 \cdot (-1)}{2 - (-1)} = \frac{4}{3} \approx 1.333.$$

Example: $\sqrt{2}$ by false position, same f and bracket

n	a	b	c	$ c - \sqrt{2} $
1	1.000000	2	1.333333	8.1×10^{-2}
2	1.333333	2	1.400000	1.4×10^{-2}
3	1.400000	2	1.411765	2.4×10^{-3}
4	1.411765	2	1.413793	4.2×10^{-4}
5	1.413793	2	1.414141	7.2×10^{-5}
6	1.414141	2	1.414201	1.2×10^{-5}

- ▶ Error shrinks by $\rho \approx 0.17$ each step: **linear**, but faster than bisection here.
- ▶ The endpoint $b = 2$ **never moves**: the bracket width tends to $2 - \sqrt{2} \approx 0.59$, not to 0.
- ▶ Error is below 10^{-6} after 8 steps (bisection: 19).

When false position is slow: the stuck endpoint

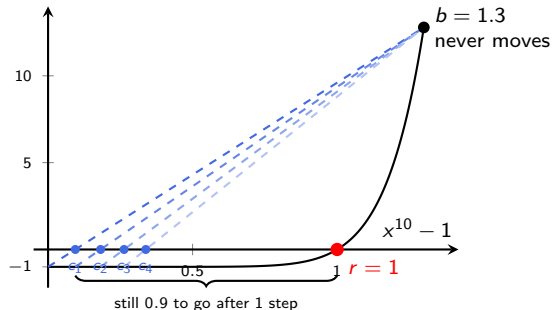
$f(x) = x^{10} - 1$ on $[0, 1.3]$, root $r = 1$.

- ▶ f is very flat on the left and steep on the right, so every secant line crosses the axis far to the left of r .
- ▶ $f(c_k) < 0$ every time, so a moves and $b = 1.3$ is never replaced: steps of only ≈ 0.08 .

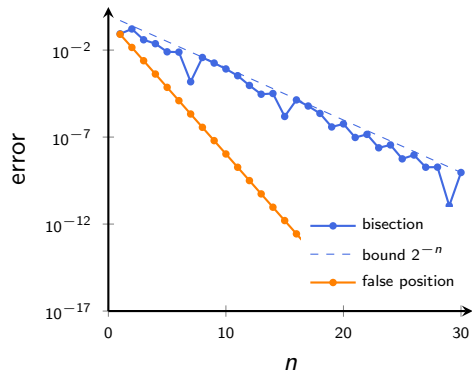
method	steps to 10^{-6}
bisection	20
false position	61

Lesson

"Smarter" is not always faster: one stuck endpoint can make false position much slower than bisection.



Bisection vs. false position



$x^2 - 2$ on $[1, 2]$. Both are straight lines on semilogy:
linear convergence.

	bisection	false position
needs a bracket	yes	yes
always converges	yes	yes
rate (linear)	$\rho = \frac{1}{2}$	ρ depends on f
error bound in advance	yes	no
bracket width $\rightarrow 0$	yes	not always
f -evaluations per step	1	1

Bracketing = reliable, not fast

Neither method can fail once you have a bracket. The price is speed: both converge only linearly.

When do we stop?

Three common tests (use more than one):

1. **Bracket width:** $(b - a)/2 < \text{tol.}$

Gives a true error bound for bisection, but may **never** trigger for false position.

2. **Step size:** $|c_n - c_{n-1}| < \text{tol.}$

Works for both; not a guarantee.

3. **Residual:** $|f(c)| < \text{tol}_f.$

Depends on the scale of f : compare f and $10^6 f$.

Always add a **maximum number of iterations**.

Floating point floor

Below $\text{tol} \approx 10^{-16} |r|$ there are no more floats between a and b : the midpoint equals a or b , and the loop runs forever without `maxit`.

Practical recipe

- ▶ Plot f ; choose a bracket with a sign change.
- ▶ Pick a **relative** tolerance, e.g. $10^{-10} |c|$.
- ▶ Stop on width or step size, cap with `maxit`.

Summary

Bracketing

- ▶ root: $f(r) = 0$
- ▶ bracket: $f(a)$, $f(b)$ of opposite sign, f continuous
- ▶ the bracket always holds a root

Bisection

- ▶ $c = a + (b - a)/2$
- ▶ error $\leq (b - a)/2^n$
- ▶ linear, $\rho = \frac{1}{2}$
- ▶ steps = $\lceil \log_2((b - a)/\text{tol}) \rceil$

False position

- ▶ c = zero of the secant line
- ▶ linear, ρ depends on f
- ▶ often faster, can be much slower
- ▶ one endpoint may get stuck

Both: guaranteed to converge, one f -evaluation per step, limited by machine epsilon.

Next lecture: methods that do not need a bracket.

Fixing the stuck endpoint: the Illinois method

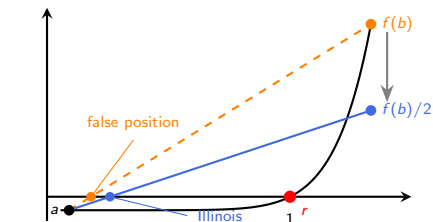
Illinois = false position + one rule

If the same endpoint is kept in **two steps in a row**, halve its stored function value (e.g. $f(b) \leftarrow f(b)/2$) before the next secant step.

One step:

1. $c = \frac{af(b)-bf(a)}{f(b)-f(a)}$ as in false position; compute $f(c)$.
2. Replace the endpoint whose f has the sign of $f(c)$.
3. Same side replaced as in the last step (track side $= \pm 1$)? Halve the stored f of the *other* endpoint.

Why it is safe: halving changes the *size* of $f(b)$, not its *sign*: $[a, b]$ stays a bracket. Still one f -evaluation per step; the convergence becomes **superlinear** (order ≈ 1.44).



$x^{10} - 1$, step 3: the halved $f(b)$ tilts the line and pushes c towards r .

$x^{10} - 1$ on $[0, 1.3]$ steps to 10^{-6}	
bisection	20
false position	61
Illinois	13

You will implement it in the notebook.